

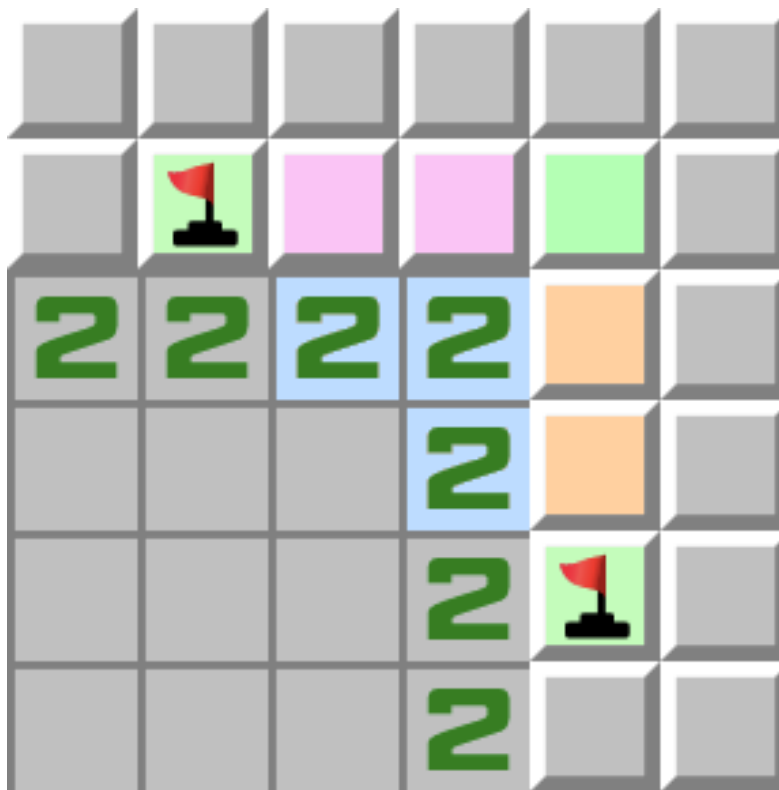
Software Development Plan (SDP)

Project: MiniSweeper

Course: הנדסת תוכנה ואבטחת מידע

Standard: IEEE 1058 – Software Project Management Plan

ElephantsTeam



1. Overview

1.1 Project Summary

The MiniSweeper project is a two-player desktop game developed in Java, following the Model-View-Controller (MVC) architecture. The objective of this project is to design, implement, and test a functional and interactive version of the classic Minesweeper game, enriched with quiz-based challenges and scoring logic. The game is designed for offline use and stores its data in CSV files. The development will follow an iterative approach divided into three main cycles (iterations).

The project aims to demonstrate proper application of software engineering principles, including modular design, object-oriented programming, agile development practices, version control through GitHub, and adherence to academic standards. The main deliverables include the working MiniSweeper game, a complete documentation set, and supporting materials.

1.1.1 Purpose

The purpose of the MiniSweeper project is to design and implement an interactive Minesweeper-style game that supports two players, multiple difficulty levels, and educational quiz elements. The system aims to provide a complete learning experience where users engage in logical thinking while also answering questions to gain or lose points. The development also serves as a practical exercise in applying software engineering principles taught in the course.

1.1.2 Scope of the Product

The product will provide: A playable Minesweeper board with difficulty levels (Easy, Medium, Hard). A turn-based two-player mode. Quiz questions that appear during gameplay. A scoring and lives system. Game history storage using CSV files. A simple and intuitive graphical user interface (GUI). The system will run on Windows, macOS, and Linux platforms with Java 17+ installed. No server connection or external database is required.

1.1.3 Objectives and Success Criteria

The primary objectives of the project are: Deliver a functioning MiniSweeper game that follows the SRS requirements. Provide a user-friendly interface with clear game feedback. Ensure data is stored and retrieved correctly through CSV files. Complete all development work within three planned iterations. Produce full documentation including SRS, SDP, testing, and final delivery. The project is considered successful when all functional and non-functional requirements in the SRS are met, and the game runs without critical errors.

1.1.4 Constraints

The main project constraints include: A limited academic timeline of three development iterations. Team members must balance academic workload with the project schedule. The system must be implemented entirely in Java, following course guidelines. Only local CSV files may be used for storage (no databases allowed). The game must run offline.

1.1.5 Assumptions

The project assumes: All team members have access to Java development tools such as IntelliJ or Eclipse. The end user (player) has basic knowledge of Minesweeper rules. The system will be tested on standard laptops running Windows or macOS. CSV files will be accessible, readable, and writable on the user's machine.

1.1.6 Deliverables

The deliverables for this project include: MiniSweeper executable (JAR). Full source code hosted in a GitHub repository. Software Requirements Specification (SRS). Software Development Plan (SDP). Design diagrams (UML). Test documents and quality assurance reports. Final project presentation.

1.1.7 Evolution of This Document

This Software Development Plan (SDP) is a living document. It will be updated throughout development as new information emerges, requirements evolve, or risks change. Each iteration may introduce adjustments to planning, responsibilities, or development focus. All updates will be documented in the GitHub repository.

2. References

1. IEEE Std 1058 – Standard for Software Project Management Plans.
2. IEEE Std 830 – Recommended Practice for Software Requirements Specifications.
3. MiniSweeper Project Requirements Document (V1.2).
4. SRS_elephantsTeam.

3. Definitions

MVC – Model View Controller, an architectural pattern separating the application into three components.

CSV – Comma-Separated Values, a text format used for storing tabular data.

FR – Functional Requirement.

SDP – Software Development Plan.

GUI – Graphical User Interface, a visual interface that lets the user interact with the system.

UML – Unified Modeling Language, a standard set of diagrams used to model software systems.

JAR – Java ARchive, a packaged file containing compiled Java code to run the application.

5. Managerial Process Plan

5.2 Work Plan

5.2.1 Work Activities

Each iteration lasts approximately three weeks. Iteration 1 focuses on core logic, Iteration 2 on interface and scoring, and Iteration 3 on question handling, history, and integration testing.

Iterations table shown in the next page.

Iteration	Goal	User Story
Iteration 1	Implement basic game logic (board, player turns, and cell interaction).	As a player, I want to click a cell to see what is inside, so I can play the game. As a player, I want to mark a cell with a flag, so I can remember where I think a mine is. As a player, I want empty cells to open automatically, so I don't have to click them one by one.
Iteration 2	Implement GUI and scoring system.	As a player, I want to see a clear game board on the screen, so I understand where to click. As a player, I want to see my score and lives, so I know how I am doing during the game. As a player, I want the game to tell me if I won or lost, so I know the result.
Iteration 3	Finalize advanced features and testing.	As a player, I want to see a list of my previous games, so I can track my performance. As a user, I want the game to save my data in files, so my progress is not lost.

Iteration	Requirement / User Story	Tasks	Executors
Iteration 1	As a player, I want to click a cell to see what is inside, so I can play the game.	1. Create the data structure for the board and cells. 2. Implement the reveal logic (mine, number, empty). 3. Connect the reveal logic to the UI so clicks work correctly. 4. Acceptance Test	1.Asil 2.Juna 3.Reem 4.Weaam (Product Owner)
Iteration 1	As a player, I want to mark a cell with a flag, so I can remember where I think a mine is.	1. Add flag status to the Cell class. 2. Implement right-click or flag toggle behavior. 3. Add visual flag indicator to the UI. 4. Acceptance Test	1.Weaam 2.Reem 3.Asil 4.Weaam (Product Owner)
Iteration 1	As a player, I want empty cells to open automatically, so I don't have to click them one by one.	1. Implement the cascade (recursive reveal) algorithm. 2. Test cascade logic on multiple board sizes. 3. Connect cascade behavior to the UI for visual updates. 4. Acceptance Test	1.Juna 2.Asil 3.Reem 4.Weaam (Product Owner)

5.2.2 Schedule Allocation

Iteration	Duration (Weeks)	Planned Completion
Iteration 1	Weeks 1–3	Nov 16 – Dec 7
Iteration 2	Weeks 4–6	Dec 8 – Dec 29
Iteration 3	Weeks 7–9	Dec 30 – Jan 20

5.3 Control Plan

5.3.1 Requirements Control Plan

All requirements will be version-controlled using GitHub. Changes to requirements can only be approved by the Product Owner after discussion with the Scrum Master. Each change will be documented in a change log with date, author, and rationale.

5.3.4 Quality Control Plan

Quality will be ensured through regular peer reviews, iterative testing, and compliance with coding standards. Each iteration includes internal verification, and the final product will undergo functional, usability, and performance testing.

5.4 Risk Management Plan

Risk	Mitigation Strategy	Responsible Member
Delay in completing GUI implementation.	Divide tasks early and set internal deadlines.	Asil
Data file corruption or CSV read errors.	Implement validation and auto-recovery mechanism.	Reem
Integration issues between logic and interface.	Schedule early integration testing after Iteration 2.	Weaam
Loss of GitHub repository access or version conflicts.	Use branching workflow and regular commits by all members.	Juna

6. Technical Process Plan

6.1 Process Model

The project uses an Agile development approach implemented through the Scrum methodology. Development is organized into three iterations, each guided

by Scrum roles: Product Owner, Scrum Master, and Developers. Each iteration produces a working increment of the MiniSweeper system.

6.2 Methods, Tools and Techniques

Development will use Java (JDK 17) within IntelliJ IDEA. Version control is managed via GitHub. Testing will use JUnit. Documentation will be written in Word and exported as PDF. The MVC pattern ensures logical separation between data, UI, and control.

7. Supporting Process Plan

7.2 Verification and Validation Plan

Verification will ensure that each module meets its design and requirement specifications. Validation will confirm the system fulfills user needs. Test cases will be derived from the SRS and executed after each iteration. Acceptance testing will be performed by the Product Owner using realistic game scenarios.

7.3 Documentation Plan

All project documentation, including SRS, SDP, design diagrams, test plans, and user manual, will be maintained in the team's GitHub repository. Each document will include version tracking and change logs. The final documentation will be compiled and submitted along with the project source code and executable file.

Team Work Division

Weaam: Main backend developer, responsible for game logic and data handling sections of SRS and SDP, **Product Owner**.

Asil: Frontend developer, responsible for GUI design, iteration planning, and formatting of documentation, **Scrum Master**.

Reem: Frontend and integration developer, in charge of testing and quality assurance documentation.

Juna: Backend developer, responsible for risk analysis, GitHub management, and overall document editing.

Members Contribution to the SDP

Weaam (Product Owner):

Overview (1), Work Plan (5.2), Iteration definitions, Task breakdown, Requirements clarification & feature prioritization.

Asil (Scrum Master):

Process Model (6.1), Quality Control Plan (5.3.4), Reviewing Work Plan (5.2), GitHub repository setup, Final editing.

Reem:

Risk Management (5.4), Requirements Control (5.3), Documentation structure.

Juna:

Methods & Tools (6.2), References, Definitions, UI prototype assistance.

Group Collaboration

All team members participated in:

UI prototype decisions

Requirements verification

Review and approval of the final SDP document.